CRISP-DM Plan for FraudGuard: Imbalanced Credit Card Fraud Detection

A beginner-to-intermediate data science portfolio project

---

1. Business Understanding

Objective: Build a machine learning system that flags likely fraudulent credit card transactions in near real time, so a bank or payments provider can block or review them before money is lost, while keeping false alarms on genuine customers low enough to stay usable.

Problem: Card fraud is extremely rare relative to total transaction volume — in this dataset, fraudulent transactions make up only 0.17% of all records (492 out of 284,807). This severe class imbalance means a naive model can score 99.8% accuracy while catching zero fraud, so the real technical challenge is building and evaluating a model that performs well on the rare class specifically, not on the dataset as a whole.

Market Context: Global card fraud losses are estimated in the tens of billions of dollars annually, and the problem is directly relevant to Kenya's rapidly growing digital payments ecosystem (M-Pesa, bank cards, merchant POS systems), where fraud detection increasingly underpins consumer trust in cashless transactions — the same trust dynamic that affects small merchants like Shawarma Mtaani when accepting mobile or card payments.

Target Audience: Banks and card issuers (fraud operations teams), payment processors, e-commerce platforms, and fintech risk teams who need an automated first-pass filter before manual review.

Success Criteria

- **Business:** Maximize fraud caught (recall) while keeping false positives low enough that genuine customers are not routinely blocked or annoyed — framed as a precision/recall tradeoff a fraud team can tune.
- **Technical:** Beat a naive baseline ("predict no fraud always") using Precision-Recall AUC (PR-AUC) rather than accuracy, since PR-AUC is the honest metric on a 0.17% positive-class dataset.
- **Impact:** Produce a reusable, well-documented pipeline (EDA → imbalance handling → model comparison → business-framed evaluation) that demonstrates both data science skill and understanding of fraud/security context.

2. Data Understanding

Data Source: The ULB Credit Card Fraud Detection dataset (Kaggle, mlg-ulb/creditcardfraud) — 284,807 anonymized European card transactions collected over two days in September 2013, with 492 confirmed frauds.

Data Characteristics

- **Time** — seconds elapsed since the first transaction in the dataset.
- **Amount** — transaction value.
- **V1–V28** — 28 numerical features already transformed via PCA to protect the identity of cardholders and merchants; individual columns are not human-interpretable.
- **Class** — the target label: 0 = normal, 1 = fraud.
- No missing values across all 284,807 rows and 31 columns.

Exploration

- Class distribution: 284,315 normal vs. 492 fraud (0.1727% positive rate) — confirms this is a severe imbalance problem, not a routine classification task.
- Fraudulent transactions have a higher average amount (~122) than normal ones (~88), but with substantial overlap in distribution, so amount alone is a weak signal.
- Time-of-day patterns show fraud is not uniformly distributed across the 24-hour cycle, hinting that engineered time features could add signal.

3. Data Preparation

Objective: Get the raw transaction table into a clean, properly scaled, and correctly split form — while being careful not to leak information between train and test sets.

Preprocessing pipeline

- **Scaling:** Standardize *Amount* and *Time* with StandardScaler, since the V1–V28 features are already on a comparable PCA-transformed scale.
- **Train/test split:** 80/20 split using stratification on the Class label, ensuring the same ~0.17% fraud ratio is preserved in both sets.
- **Imbalance handling (train set only):** Two approaches are compared — class-weighting (class_weight='balanced' / scale_pos_weight) which re-weights the loss function without touching the data, and SMOTE (Synthetic Minority Oversampling), which generates synthetic fraud examples by interpolating between real ones. SMOTE is applied only to training data to avoid leaking synthetic patterns into evaluation.

Output: A single clean, scaled table (X_train, X_test, y_train, y_test) plus a SMOTE-augmented training variant, ready for modeling.

4. Modeling

Objective: Compare several models of increasing complexity, each combined with different imbalance-handling strategies, to find the best precision/recall tradeoff.

Model A — Baseline (sanity check)

A "predict normal always" dummy classifier, used purely to demonstrate why accuracy is a misleading metric on this dataset (99.8% accuracy, 0% fraud recall).

Model B — Logistic Regression

A simple linear baseline, run three ways: with no imbalance handling, with class_weight='balanced', and on the SMOTE-augmented training set — to isolate the effect of each imbalance-handling technique on the same underlying model.

Model C — Random Forest

An ensemble of decision trees using class_weight='balanced', offering a nonlinear decision boundary and generally stronger precision than linear models on this type of data.

Model D — XGBoost

A gradient-boosted tree ensemble using scale_pos_weight to account for the imbalance, with each new tree focused on correcting the previous trees' mistakes. Typically the strongest performer on structured/tabular fraud

data of this kind.

Frameworks: pandas, numpy, scikit-learn, imbalanced-learn (SMOTE), xgboost, matplotlib/seaborn for visualization.

5. Evaluation

Objective: Judge every model on metrics appropriate for a 0.17% positive-class problem, not on accuracy.

Metrics used

- **Precision** — of all transactions flagged as fraud, what fraction were actually fraud (controls false-alarm rate on genuine customers).
- **Recall** — of all actual fraud cases, what fraction were caught (controls missed fraud / money lost).
- **PR-AUC (Average Precision)** — the primary ranking metric across models, since it is far less inflated by the huge true-negative class than ROC-AUC on this kind of imbalance.
- **Confusion matrix** — translated into plain business language: true positives caught, false positives (annoyed genuine customers), false negatives (fraud that slipped through).

Results summary (from executed notebook)

- Naive baseline: 99.8% accuracy, 0% recall — confirms accuracy alone is unusable here.
- Logistic Regression (plain): precision ≈ 0.83, recall ≈ 0.64.
- Logistic Regression (balanced / SMOTE): recall rises to ≈ 0.92, but precision drops sharply (≈ 0.06) — many false alarms for each fraud caught.
- Random Forest (balanced): precision ≈ 0.96, recall ≈ 0.76 — very few false alarms, but misses more fraud.
- XGBoost (scale_pos_weight): best overall balance — PR-AUC ≈ 0.875, precision ≈ 0.89, recall ≈ 0.83.

Success Criteria: Every real model must beat the naive baseline on recall while keeping precision at a level a fraud-ops team could realistically act on; XGBoost meets this bar most convincingly and is selected as the final model.

6. Deployment

Objective: Package the work as a demoable, reusable artifact rather than a one-off script.

MVP

- A documented Jupyter notebook (fraud_detection_project.ipynb) covering the full pipeline from raw CSV to final model comparison, runnable end-to-end in VS Code or Google Colab.
- A README summarizing the business problem, key insight (accuracy is misleading), and final model metrics in plain language, published to GitHub (Victorwa05) as a portfolio piece.

Stretch goals

- Wrap the final XGBoost model in a lightweight FastAPI endpoint (/predict-fraud) that accepts a transaction and returns a fraud probability.
- Add a simple Streamlit front-end where a reviewer can paste transaction values and see the model's prediction and confidence.
- Tune the classification threshold interactively, showing how precision and recall trade off in real time — a strong talking point in interviews.

Why this is a strong beginner portfolio project

- Uses a real, well-known, industry-relevant dataset rather than a toy problem.
- Teaches the single most important lesson in applied classification: why accuracy fails on imbalanced data, and what to use instead.
- Touches several imbalance-handling techniques (class weighting, SMOTE) and model families (linear, ensemble, boosting) in one coherent pipeline.
- Connects naturally to your cybersecurity background (ABSA Bank, Salaam Technology) — you can speak to both the fraud/security context and the modeling approach in interviews.
- Fully reproducible: dataset, notebook, and results are all self-contained and ready to push to GitHub as-is.